

# When Does Streaming Tool Use Help?

Characterizing Tool-Intent Stabilization in Streaming  
Retrieval-Augmented Generation

Elroy Galbraith, PhD  
SMG Labs

June 18, 2026

## Abstract

Streaming Retrieval-Augmented Generation (Streaming RAG) reduces user-perceived latency by issuing tool queries in parallel with ongoing user input, before the utterance is complete. Reported gains are aggregate, yet the mechanism’s benefit is fundamentally *query-intrinsic*: speculation can only help when the correct tool query becomes determinable before the user stops speaking or typing. We isolate and measure this property—**tool-intent stabilization**, the point in the input stream at which a speculative query’s retrieval converges to the answer-bearing result. On the CRAG benchmark (1371 validation questions) we (i) measure the distribution of stabilization, (ii) derive a model-agnostic bound  $H$  on the portion of tool latency that can be hidden behind the user’s remaining input, as a function of tool latency  $L$  and input cadence  $\delta$ , (iii) validate against a working streaming pipeline that realized savings meet or exceed this bound, and (iv) identify which query properties predict early versus late stabilization. The study requires no model training and runs on commodity CPU hardware. We find that sufficiency stabilizes early on the groundable, BM25-retrievable subset of the benchmark (mean  $\phi_{\text{suf}} = 0.26$  over the 21.3% of questions where gold evidence is both verbatim-present and retrievable), that 73.9% of queries admit substantial latency hiding at a realistic operating point—95.2% of the retrieved-gold subset and 68.1% of the remainder—and that question type is an ordered predictor of stabilization (Kruskal–Wallis  $p = 0.017$ ), directly informing when a learned speculative trigger is worth its cost.

## 1 Introduction

Tool-augmented dialogue systems increasingly rely on external retrieval to remain factual, but tool calls add latency that disrupts conversational flow. Streaming RAG [1] addresses this by predicting and issuing tool queries *in parallel with* the user’s still-arriving input, then reflecting on whether retrieved results suffice. The approach reports sizable accuracy and latency improvements and is modality-agnostic, applying equally to typed input.

The reported latency benefit, however, is an aggregate over a benchmark. It leaves a basic question unanswered: **for which queries can speculation help at all, and by how much?** The answer is not a property of the model, the retriever, or the speech stack—it is a property of *where, within an utterance, the information that determines the tool query first appears*. If the decisive term arrives last (“who makes the console called the *PlayStation*”), no early speculative query can retrieve the right evidence and the latency win collapses to zero regardless of system quality. If the decisive term arrives early, almost the entire tool latency can be hidden behind the remaining input.

We study this directly. By characterizing tool-intent stabilization as a measurable, query-intrinsic quantity, we obtain (i) a model-agnostic bound on the share of tool latency that can be hidden

behind ongoing input, computable *before* any system is built; (ii) a principled account of the failure mode; (iii) actionable guidance on whether a learned trigger is worth its cost; and (iv) results that transfer across text and speech because the quantity is modality-agnostic. Our contribution is measurement and analysis rather than a new system.

## 2 Background and Related Work

Retrieval-Augmented Generation [4] grounds generation in retrieved evidence; dense passage retrieval [2] and the sparse BM25 model [5] are the standard retrieval backbones. Streaming RAG [1] adds a speculative dimension: a *Trigger* decides when to fire a tool query from a partial utterance, multiple speculative *Threads* run in parallel, and a *Reflector* judges whether results suffice. The idea is closely related in spirit to speculative decoding [3], which hides latency by doing work that may be discarded; here the discarded work is a premature tool call. Our analysis is orthogonal to the system: we quantify the headroom that any such system can exploit, set by the query alone.

## 3 Problem Formalization

Let a query  $Q$  be a word sequence  $q_{1:n}$  of length  $n$  words, and let  $q_{1:t}$  denote the prefix of the first  $t$  words (we measure stabilization in words throughout, matching the whitespace tokenization used by both the retriever and the input-cadence model below). Let  $R$  be a retriever returning a ranked document list; write  $d_t = \text{top-1}(R(q_{1:t}))$  for the top document retrieved from the prefix,  $d_n$  for the full-query top document, and  $d^*$  for the gold answer-bearing document.

**Self-consistency stabilization**  $t_{\text{sc}}$ . the smallest  $t$  such that for all  $s \geq t$ ,  $d_s = d_n$ : the prefix already retrieves what the full query will.

**Sufficiency stabilization**  $t_{\text{suf}}$ . the smallest  $t$  such that  $d^* \in \text{top-}k(R(q_{1:t}))$ : the prefix already surfaces the gold evidence. This is the operationally meaningful notion, since the full query itself may retrieve imperfectly.

**Stabilization fraction**  $\phi = t^*/n \in (0, 1]$ , reported for both  $t_{\text{sc}}$  and  $t_{\text{suf}}$ . Lower  $\phi$  means earlier stabilization and more headroom to hide latency.

**Stabilization volatility**  $V$ , the number of top-1 changes occurring *after*  $d_n$  is first reached. High  $V$  signals early-commit risk: a confident-looking early result that a later token would overturn.

**Hidden latency.**

$$H(Q; L, \delta) = \min\left(L, \max\left(0, \frac{n-t^*}{\delta}\right)\right), \quad (1)$$

where  $L$  is tool latency (ms) and  $\delta$  is the input cadence in words per second, so that  $(n - t^*)/\delta$  is the residual input time (in seconds, scaled to ms) still to arrive after the query has stabilized.  $H$  is the portion of tool latency that completes behind the user’s remaining input. It bounds the *input-hideable* component of the per-query perceived-latency saving—not the total saving, since a real pipeline can additionally overlap other costs (§5.4). A slower cadence (smaller  $\delta$ ) lengthens the residual and so raises  $H$  when tool latency is the larger term.

**Streamable.**  $\text{Streamable}(Q; L, \delta, \theta)$  holds iff  $H \geq \theta L$  for a coverage threshold  $\theta$  (e.g.  $\theta = 0.8$  hides at least 80% of tool latency). The streamable fraction of a workload is the central deployment-relevant statistic.

**Hypotheses.** The study is organized around four pre-registered hypotheses, each mapped to a research question (RQ) below.

**H1 (RQ1).** The distribution of  $\phi$  is bimodal: a large early-stabilizing mass (decisive entity stated up front) plus a tail that stabilizes only near the end (decisive term last).

**H2 (RQ2).** For large tool latency  $L$ , the binding constraint on  $H$  is the residual input time  $(n - t^*)/\delta$  rather than  $L$ ; consequently a *slower* input cadence  $\delta$  paradoxically *raises* the hideable fraction.

**H3 (RQ4).** CRAG question type predicts  $\phi$ : simple lookups stabilize early, while comparison, aggregation, multi-hop, and post-processing queries stabilize late.

**H4 (robustness).** Switching from sparse (BM25) to dense retrieval shifts absolute  $t^*$  but preserves the ordering of  $\phi$  across types.

We test H1–H3 directly (§5); the dense-retriever condition for H4 is left to future work, and we instead report a top- $k$  robustness sweep (§4, Table 3) as a within-retriever sensitivity check.

## 4 Method

**Data.** We use CRAG, the Comprehensive RAG Benchmark [6], Task 1&2 dev set: 2706 questions, of which 1371 are the validation split we analyze. Each question carries up to five retrieved web pages, a gold answer (plus alternates), and a question-type label.

**Grounding  $d^*$ .** CRAG provides no gold-*passage* label, only gold answer strings. We derive  $d^*$  by grounding the normalized gold answer(s) into passage text: substring match for multi-token answers, and word-boundary match for single-token answers, over the answer and its alternates. Questions whose answer is never a literal span (false-premise, “I don’t know”, and many aggregation/dynamic answers) are *ungroundable* and excluded from  $t_{\text{suf}}$ ; we report the groundable rate explicitly. Because  $t_{\text{sc}}$  requires no grounding, we report it as a grounding-free robustness check alongside  $t_{\text{suf}}$ .

String grounding is a recall-oriented heuristic and can produce false positives: a single-token gold answer (a year, a number, “yes”/“no”) may match a passage *coincidentally*, registering sufficiency before the genuinely answer-bearing evidence arrives and biasing  $t_{\text{suf}}$ —and thus  $\phi_{\text{suf}}$ —*early*. Multi-token answers, which require a contiguous substring match, are far less prone to this. We therefore read  $\phi_{\text{suf}}$  as a lower bound on stabilization time on the groundable subset, and treat the early-stabilization finding as directionally robust rather than exact.

**Passages and retrieval.** Page HTML is cleaned to text and chunked into overlapping 120-word windows. We index each question’s passages with BM25 [5] and retrieve over every prefix  $q_{1:t}$ ,  $t = 1 \dots n$ , recording  $d_t$  and the top- $k$  set at each prefix. We sweep  $k \in \{1, 3, 5\}$  for the sufficiency definition.

**Latency model and harness (RQ3).** We stream each query as a uniform word sequence—a controlled proxy for input cadence  $\delta$ —and sweep  $(L, \delta, \theta)$  analytically over the grid  $L \in \{100, 300, 600, 1000\}$  ms,  $\delta \in \{2, 3, 4\}$  w/s,  $\theta \in \{0.5, 0.8, 1.0\}$  to obtain the streamable surface (RQ2). For RQ3 we replay a subset of questions through a faithful asynchronous Streaming RAG harness (fixed-interval Trigger, parallel Threads, Reflector) calibrated with the per-stage latencies reported by Arora et al. [1] (Table 3): query generation  $\approx 590$  ms and fuse/reflect  $\approx 2520$  ms, in addition to the swept tool latency  $L$ . We compare the *measured* perceived-latency saving (baseline minus streaming) against the  $H$  bound.

**Reproducibility.** The pipeline is training-free and CPU-only. We release the harness code together with the derived  $d^*$  passage labels (the only non-trivial artifact, since CRAG ships none), so that every per-question metric and the full  $(L, \delta, \theta)$  grid can be recomputed offline without re-running retrieval.

## 5 Results

We analyze the 1371 validation questions of CRAG Task 1&2. Under our string-grounding procedure, 35.4% of questions are *groundable* (the gold answer appears verbatim in the retrieved pages); the remainder—dominated by false-premise items and answers that never occur as a literal span (many aggregation and dynamic questions)—are excluded from sufficiency statistics. At  $k=3$ , BM25 surfaces the gold passage in the top- $k$  at some prefix for 21.3% of all questions. All reported  $\phi_{\text{suf}}$  statistics are conditional on this retrieved-gold population;  $\phi_{\text{sc}}$  is defined for every question and serves as a grounding-free check.

This retrieved-gold subset is not a random sample of the benchmark: it is the intersection of questions whose answer is verbatim-groundable *and* whose gold passage BM25 can surface from some prefix. Both filters plausibly select for queries with a salient, early lexical anchor, so the conditional  $\phi_{\text{suf}}$  statistics should be read as characterizing this favorable slice, not “most answerable questions”; if anything, the selection biases the reported stabilization *earlier* than it would be on the full workload. We therefore lean on  $\phi_{\text{sc}}$ , defined on all 1371 questions, as the grounding-free companion to every sufficiency claim.

### 5.1 RQ1: How is stabilization distributed?

The two stabilization notions behave very differently (Fig. 1). Self-consistency stabilizes *late*: the lexical top-1 keeps changing until, on average,  $\phi_{\text{sc}}$  reaches mean 0.67 (median 0.73)—roughly three-quarters of the way through the query. Sufficiency, in contrast, stabilizes *early*: the gold passage enters the top- $k$  at mean  $\phi_{\text{suf}} = 0.26$  and median 0.14. The gap is the central empirical observation of this study: **the answer-bearing document is retrievable from a short prefix long before the query’s lexical top-1 settles**. For streaming tool use this is the favorable regime—one need not wait for the utterance to “finish” for the right evidence to be in hand.

The  $\phi_{\text{suf}}$  distribution (Fig. 1) is right-skewed with a large mass near zero and a thinner late tail, rather than cleanly bimodal as H1 anticipated. Volatility is modest: the post-stabilization top-1 changes at all for only 20.9% of questions (mean  $V = 0.52$ ), so early-commit risk is concentrated in a minority of queries rather than pervasive.

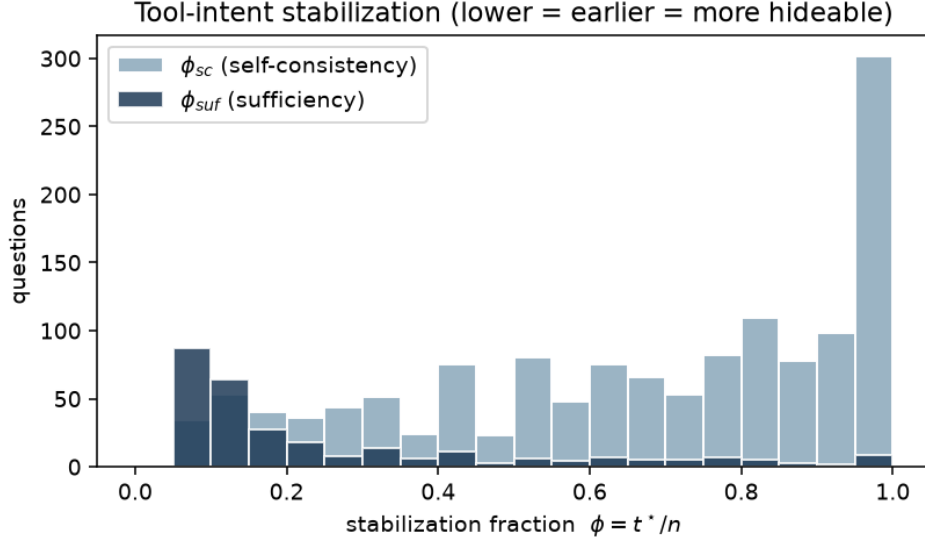


Figure 1: Distribution of stabilization fraction  $\phi$  at  $k=3$ . Sufficiency ( $\phi_{\text{suf}}$ , dark) concentrates near zero—gold evidence is retrievable early—while self-consistency ( $\phi_{\text{sc}}$ , light) is late and broad.

## 5.2 RQ4: What predicts early vs. late stabilization?

Question type is an ordered predictor of  $\phi_{\text{suf}}$ , spanning a roughly threefold range (Table 1, Fig. 2). A Kruskal–Wallis test across all eight types rejects equality of the  $\phi_{\text{suf}}$  distributions ( $H=17.0$ ,  $\text{df}=7$ ,  $p=0.017$ ); restricting to the five classes with  $n \geq 10$ —which carry the effect—the test is stronger ( $H=13.9$ ,  $\text{df}=4$ ,  $p=0.008$ ). The bootstrap 95% confidence intervals in Table 1, however, overlap substantially for adjacent types: the ordering is reliable only at the extremes (*aggregation* and *comparison* earliest, *set* latest), not as a strict cell-by-cell ranking, and the small-support types (*set*  $n=2$ , *post-processing*  $n=5$ ) have intervals too wide to place. We therefore claim a significant type-level effect with a robust early/late split, not a precise per-type order. The qualitative ordering refines hypothesis H3 rather than confirming it. H3 expected simple lookups to stabilize early and aggregation/comparison/multi-hop to stabilize late. We instead find *aggregation* and *comparison* among the *earliest*-stabilizing types, with *set* questions latest. The explanation is that sufficiency measures when the gold *document* becomes retrievable, not when the answer can be *computed*: a comparison or aggregation question often names its key entities up front, so the answer-bearing page is retrievable early even though producing the final answer requires post-retrieval reasoning. In other words, retrieval-sufficiency stabilization is governed by entity position in the utterance, which is not aligned with reasoning complexity. This entity-position account is carried by the well-populated classes (*simple*, *comparison*, *aggregation*, *simple\_w\_condition*), so it does not hinge on the noisy small- $n$  cells.

## 5.3 RQ2: What fraction of queries admit latency hiding?

Using  $t^* = t_{\text{suf}}$  where available and falling back to  $t_{\text{sc}}$  otherwise, we compute the streamable fraction over the full  $(L, \delta, \theta)$  grid (Table 2, Fig. 3). At the central operating point ( $L=600$  ms,  $\delta=3$  w/s,  $\theta=0.8$ ), 73.9% of queries admit hiding at least 80% of tool latency.

This blended figure mixes two populations and two notions of stabilization, and we report it decomposed. On the 21.3% of queries with retrieved gold—where  $t^* = t_{\text{suf}}$  reflects evidence actually

Table 1: Sufficiency stabilization by CRAG question type ( $k=3$ , retrieved-gold questions), sorted earliest to latest. The last column is a bootstrap 95% CI for the mean (10,000 resamples); intervals overlap heavily for adjacent types, so only the extreme contrasts are reliable.

| question type      | $n$ | mean $\phi_{\text{suf}}$ | 95% CI       |
|--------------------|-----|--------------------------|--------------|
| aggregation        | 32  | 0.198                    | [0.13, 0.28] |
| comparison         | 59  | 0.210                    | [0.16, 0.27] |
| multi-hop          | 15  | 0.267                    | [0.14, 0.42] |
| simple             | 118 | 0.280                    | [0.24, 0.33] |
| false_premise      | 9   | 0.289                    | [0.14, 0.48] |
| simple_w_condition | 52  | 0.307                    | [0.23, 0.40] |
| post-processing    | 5   | 0.310                    | [0.10, 0.52] |
| set                | 2   | 0.658                    | [0.62, 0.70] |

Table 2: RQ2: streamable fraction (%) at  $\theta=0.8$  over the  $(L, \delta)$  grid. Rows are tool latency  $L$  (ms); columns are input cadence  $\delta$  (words/sec). For large  $L$ , slower cadence (lower  $\delta$ ) hides more (H2).

|          | $\delta=2$ | $\delta=3$ | $\delta=4$ |
|----------|------------|------------|------------|
| $L=100$  | 82.6       | 82.6       | 82.6       |
| $L=300$  | 82.6       | 82.6       | 82.6       |
| $L=600$  | 82.6       | 73.9       | 73.9       |
| $L=1000$ | 73.9       | 63.3       | 54.5       |

in hand—95.2% are streamable; on the remaining 78.7%, where we fall back to the grounding-free  $t_{\text{sc}}$  (top-1 stopped moving), 68.1% are. The sufficiency-only number, 95.2%, is the one that matches our framing and is the honest headline for “evidence is retrievable in time”; the 73.9% blend is lower because it is dominated by the  $t_{\text{sc}}$  majority, for which “streamable” means only that the lexical top-1 has settled, not that gold is in the top- $k$ . Readers who care about evidence sufficiency should weight the 95.2% figure; those who treat top-1 stability as the deployment signal get the blend. Two further patterns stand out. First, for small tool latency ( $L \leq 300$  ms) the constraint is essentially never binding: 82.6% of queries are streamable regardless of cadence—the residual input time trivially covers a short tool call. Second, and confirming **H2**, when tool latency is large ( $L=1000$  ms) the binding constraint becomes residual input time, so a *slower* cadence *raises* the streamable fraction: from 54.5% at  $\delta=4$  w/s up to 73.9% at  $\delta=2$  w/s. Faster typists and faster talkers leave less room to hide a slow tool.

#### 5.4 RQ3: Does the bound match a working pipeline?

We replayed 60 retrieved-gold questions through the asynchronous Streaming RAG harness at  $L=600$  ms,  $\delta=3$  w/s. The measured perceived-latency saving (baseline minus streaming) averages 1122 ms, while the  $H$  bound predicts 590 ms (Fig. 4). On average, measured savings *exceed* the bound by roughly  $1.9\times$ . This is expected and informative:  $H$  models only the tool latency hidden behind input, whereas the calibrated pipeline additionally overlaps the LLM’s query-generation step ( $\approx 590$  ms [1]) with the user’s input.  $H$  is therefore a *conservative* predictor of realized savings—it captures the input-hiding component and omits other overlappable costs—so a query that  $H$  deems streamable will, in practice, save at least as much as predicted.

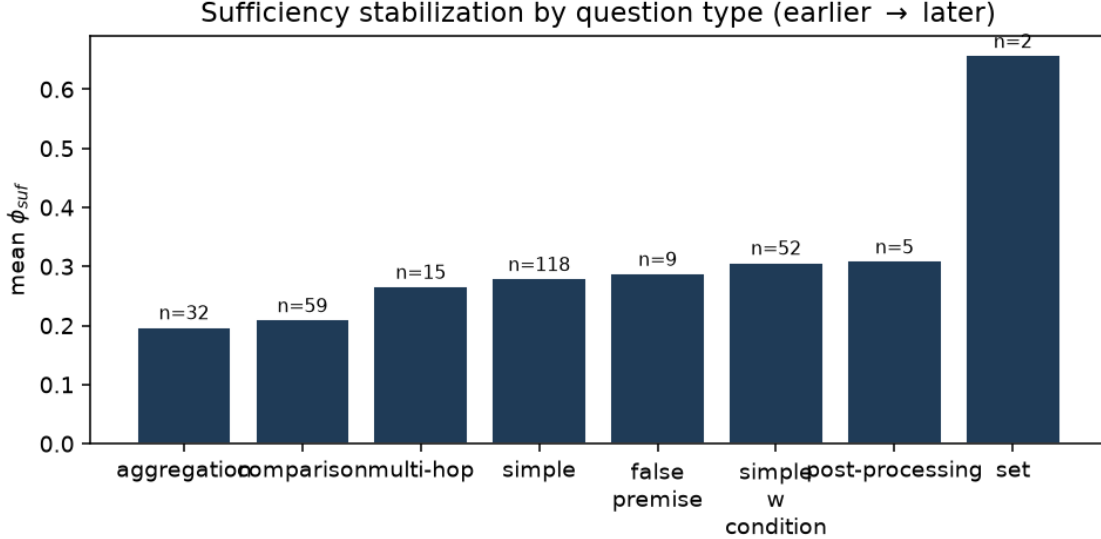


Figure 2: Mean  $\phi_{\text{suf}}$  by question type ( $k=3$ ). Type spans a  $3\times$  range, but the ordering tracks entity position, not reasoning complexity.

We are deliberately cautious about what this experiment does *not* show. At  $L=600$  ms the bound saturates: 59 of the 60 replayed queries have the same  $H=600$  ms (the input residual exceeds  $L$ ), so there is essentially no horizontal spread against which to test whether  $H$  *ranks* queries by realized saving. Consistent with this, the rank correlation between the input-hideable residual  $(n - t^*)/\delta$  and the measured saving is negligible (Spearman  $\rho = -0.06$ ). RQ3 thus validates the bound in the aggregate sense—realized savings meet or exceed it—but it does *not* establish per-query prediction; doing so requires a replay at larger  $L$  (e.g. 1000 ms), where  $H$  varies across queries (Table 2). We flag this as the primary follow-up rather than claim predictive validity here.

The few queries that save little are also instructive for volatility  $V$ . The single query with a negative saving (a 17-word comparison) is a *late*-stabilization case ( $t^*=13$ , almost no residual to hide behind) with  $V=0$ , not a high-volatility early-commit mis-fire. So in this sample  $V$  did not flag the failure, and the downside is structural:  $H$ 's  $\max(0, \cdot)$  floors the bound at zero and cannot express the negative saving a real mis-fire incurs. Whether  $V$  predicts mis-fires at scale—the actionable role it is meant to play—remains untested on a sample this size, and we leave it open.

### 5.5 Retriever robustness (top- $k$ )

Varying the sufficiency top- $k$  shifts absolute values but preserves the qualitative picture (Table 3). Larger  $k$  surfaces the gold passage for more questions (retrieved-gold rate 15.6%  $\rightarrow$  25.0%) and stabilizes slightly earlier (mean  $\phi_{\text{suf}}$  0.327  $\rightarrow$  0.248), as expected since a more permissive top- $k$  is easier to satisfy. The self-consistency curve is unchanged by  $k$  (it depends only on the top-1), and the central streamable fraction is stable near 73–74%. The early-sufficiency / late-self-consistency gap holds at every  $k$ .

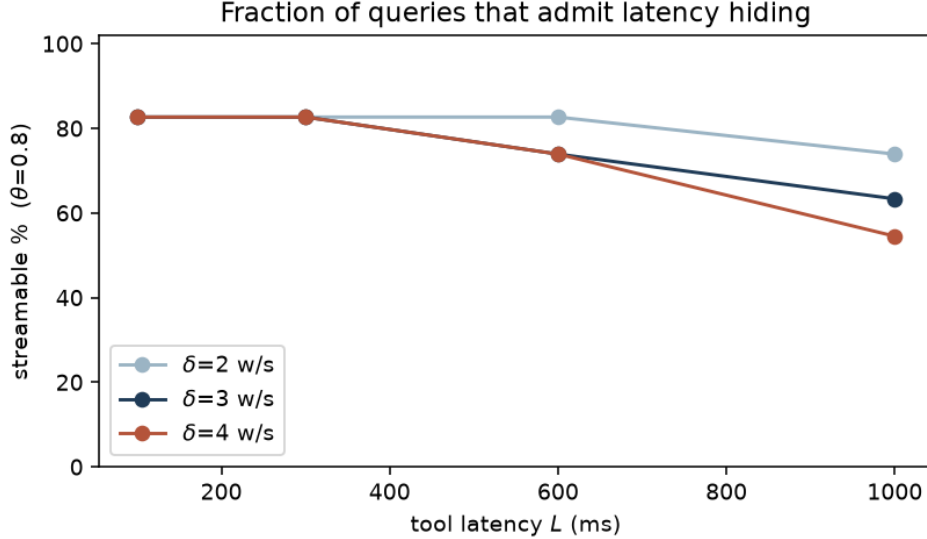


Figure 3: Streamable fraction vs. tool latency  $L$  at  $\theta=0.8$ , one line per cadence  $\delta$ . The curves separate only once  $L$  is large enough for residual input time to bind; there, slower cadence dominates (H2).

Table 3: Top- $k$  robustness.  $\phi_{sc}$  (mean 0.67 / median 0.73) is independent of  $k$  and omitted.

| $k$ | retrieved-gold | mean $\phi_{suf}$ | median $\phi_{suf}$ |
|-----|----------------|-------------------|---------------------|
| 1   | 15.6%          | 0.327             | 0.200               |
| 3   | 21.3%          | 0.264             | 0.143               |
| 5   | 25.0%          | 0.248             | 0.125               |

## 6 Threats to Validity

**Input timing.** A uniform word stream is a proxy for speech; real ASR pacing is variable and partial hypotheses get revised. We bound this here by using clean text and treat speech timing as planned follow-up.

**Stabilization measure.** Top-1 equality is a coarse proxy for “intent settled.” We mitigate by also reporting sufficiency against the gold document and by varying top- $k$ .

**Grounding.**  $d^*$  is derived by string matching, so the groundable population excludes non-verbatim answers; reported  $\phi_{suf}$  statistics are conditional on groundability, and we report  $t_{sc}$  as a grounding-free check.

**Retriever and benchmark.** BM25 is lexical and phrasing-sensitive, and results are from a single benchmark; a dense-retriever condition and a second QA set are natural extensions.



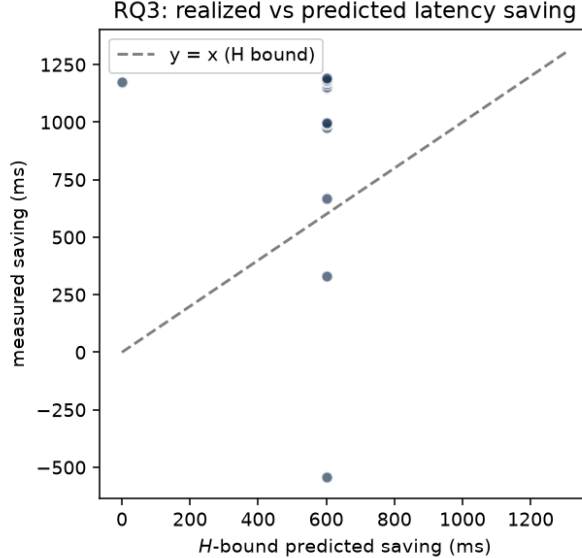


Figure 4: RQ3: per-question measured saving vs. the  $H$  bound (60 questions). Most points lie above  $y=x$  because the pipeline also hides query-generation latency, which  $H$  does not model. At  $L=600$  ms the bound saturates—59/60 points share  $H=600$  ms—so the plot demonstrates that measured savings exceed the bound, not that  $H$  ranks queries; one late-stabilizing query shows a negative saving that the floored bound cannot express.

## 7 Conclusion

Tool-intent stabilization is a query-intrinsic, model-agnostic quantity that bounds when streaming tool use can help and by how much. On CRAG, sufficiency stabilizes early on the groundable, retrievable subset and is ordered by question type (Kruskal–Wallis  $p = 0.017$ ); the  $H$  bound is conservative, with realized savings on a working pipeline meeting or exceeding it; and the streamable fraction is high at realistic operating points. Two limitations temper these claims and set the agenda for follow-up: the sufficiency results are conditional on a favorable 21.3% slice of the benchmark, and our system validation establishes conservativeness in aggregate but not per-query prediction, because the bound saturates at the operating point we replayed (§5.4). A higher- $L$  replay, a dense-retriever condition (H4), and a regression of  $\phi$  on query features are the natural next steps. Even so, the present results give system builders a principled, training-free basis for trigger design and a build/skip decision for a learned trigger.

## References

- [1] Siddhant Arora, Hamza Khan, Kaihang Sun, Xin Luna Dong, Sneha Choudhary, Seungwan Moon, Xinyao Zhang, Aman Sagar, Sai Teja Appini, Kushal Patnaik, Shubham Sharma, Shinji Watanabe, Anuj Kumar, Ahmed Aly, Yuan Liu, Florian Metze, and Zhaojiang Lin. Stream RAG: Instant and accurate spoken dialogue systems with streaming tool usage. *arXiv preprint arXiv:2510.02044*, 2025.
- [2] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense passage retrieval for open-domain

- question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [3] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [6] Xiao Yang, Kai Sun, et al. CRAG: Comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.